LAB 7 – FUZZ TESTING

Input Validation Bug Detection and Fixing using Hypothesis

This lab applies property-based fuzz testing to a Pizza Order application. Hypothesis generates unexpected inputs to test whether the application functions crash. The original implementation was tested first, the failures were examined, and defensive input validation was added before rerunning the tests.

1. Functions Under Test

Function	Purpose
toppings_price(toppings_csv)	Calculates the total price of comma-separated toppings.
pizza_inches(size)	Returns the pizza diameter for S, M or L.
split_bill(text)	Calculates each person's share from a total/people string.

2. Initial Fuzzing and Bugs Identified

The initial Hypothesis tests used `st.text() | st.none()` to generate arbitrary strings and None values. The original code failed because it assumed inputs were always valid strings and correctly formatted values. Examples observed in the failure trace included empty strings and None for toppings and pizza size, and '/', empty strings and None for bill splitting. The failures included KeyError, AttributeError and ValueError.

3. Defensive Fixes Applied

Function	Validation added
toppings_price()	Rejects non-string inputs; handles blank input; checks each topping against the allowed topping dictionary and reje
pizza_inches()	Rejects non-string inputs and validates the normalized size against S, M and L.
split_bill()	Checks input type and exact slash-separated structure; validates integer values; rejects negative totals and non-po

4. Final Test Results

The final command was `python3 -m pytest test_pizza_order.py -v`. Pytest collected all three test functions and reported 3 passed. The separate invalid-input checks also returned None for an unknown topping, invalid size, zero people, malformed bill and None input, without an uncaught exception.

5. Conclusion

The lab demonstrated how property-based fuzzing can reveal edge cases that ordinary valid-input runs do not expose. Hypothesis found failures in all three original functions. After adding defensive validation, all three fuzz tests passed and the application handled the selected invalid-input checks without crashing.

6. Execution Evidence

```
rujuljain@Rujuls-MacBook-Air Lab 7 % python3 pizza_order.py
==== Running toppings_price ====
Enter toppings as CSV (e.g. cheese,olives): cheese
Toppings Price: 40

==== Running pizza_inches ====
Enter pizza size (S/M/L): M
Size in Inches: 10

==== Running split_bill ====
Enter total and people (e.g. 900/3): 1000/3
Per Person: 333.33
rujuljain@Rujuls-MacBook-Air Lab 7 %
```

Figure 1. Original Pizza Order application execution with normal inputs.

```
=========================== short test summary info ============================
FAILED test_pizza_order.py::test_toppings_price_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_pizza_order.py::test_pizza_inches_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
FAILED test_pizza_order.py::test_split_bill_no_crash - ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
============================== 3 failed in 1.35s ===============================
```

Figure 2. Initial Hypothesis fuzzing run showing failures in all three test functions.

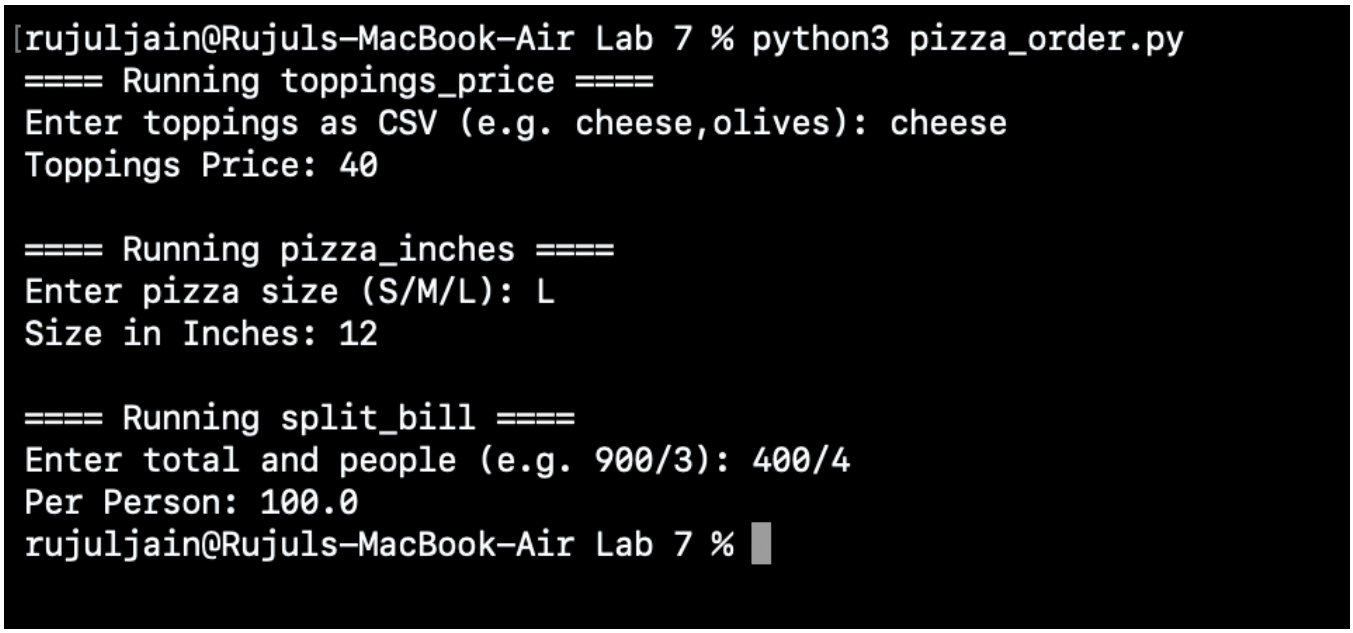


Figure 3. Corrected Pizza Order application execution.

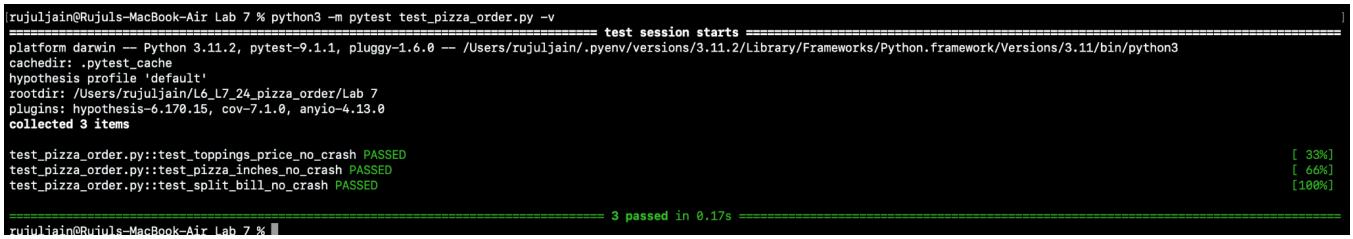


Figure 4. Final Hypothesis test run: all three tests passed.

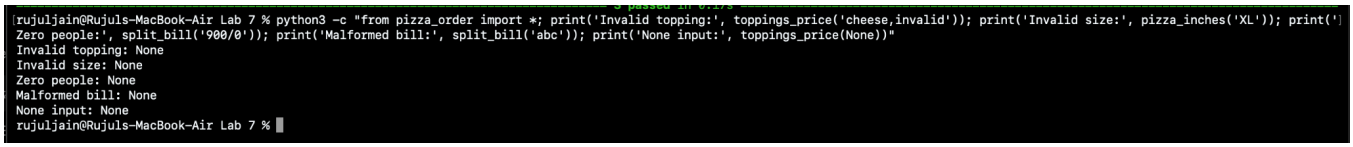


Figure 5. Manual invalid-input validation: invalid values returned None without crashing.

Lab 7 – GitHub Submission

Personal GitHub repository:

https://github.com/rujul-rj/SE-Labs-PES1UG24CS388

Lab 7 deliverables folder:

https://github.com/rujul-rj/SE-Labs-PES1UG24CS388/tree/main/Lab%207

The repository contains the corrected application script, updated Hypothesis test file, and this report with execution evidence.